

U2
-
Introducción a Docker
y
despliegues en local

2° DAW

2025/2026

Juanra Collado – jr.colladoesteve@edu.gva.es

RA's que se trabajan en esta unidad

- RA1 - Implanta arquitecturas web analizando y aplicando criterios de funcionalidad
  - Este RA también se trabaja en la U2 y U4.

- RA2 - Implanta aplicaciones web en servidores web, evaluando y aplicando criterios de configuración para su funcionamiento seguro.
  - Este RA también se trabaja en la U2, U3 y U4.

¿Qué es Docker?

- Un contenedor es una unidad de software estandarizada que empaqueta el código de una aplicación y todas sus dependencias para permitir su ejecución de manera rápida y consistente independientemente del entorno en el que se ejecute.

- Docker es una plataforma de código abierto para desarrollar, distribuir y desplegar aplicaciones en entornos aislados y seguros denominados contenedores.

¿Qué es Docker?

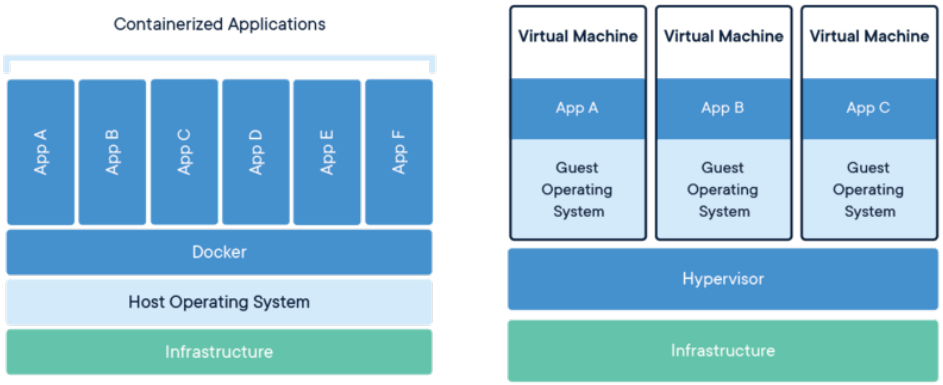
- Los contenedores tienen la característica de ser livianos, ya que no necesitan la carga adicional de un hipervisor como ocurre con las máquinas virtuales, sino que se ejecutan directamente dentro del núcleo de la máquina host.
  - Esto significa que con un determinado hardware podremos ejecutar mayor número de contenedores que si estuviéramos utilizando varias máquinas virtuales.

- Ejemplo:
  - SO Alpine (4Mb) + Web (4Kb) = Web desplegada en < 5 MB
  - Alpine es un SO Linux extremadamente ligero, segura y eficiente

¿Qué es Docker?

- Los contenedores utilizan un sistema de virtualización basado en compartición de recursos, sin virtualizar la arquitectura HW completa.

- A este tipo de virtualización se le llama OS Level Virtualization
  - https://en.wikipedia.org/wiki/OS-level_virtualization



¿Qué es Docker?

- La plataforma Docker nos permite empaquetar nuestras aplicaciones con todo lo necesario para su ejecución (librerías, código, herramientas, configuraciones), así elimina dependencias del sistema operativo de la maquina host y facilita un despliegue muy rápido de nuestras aplicaciones.
  - Ya no dependemos del SO donde estamos ejecutando y nos ahorramos problemas con librerías y configuraciones.
  - Multiplataforma

¿Qué es un contenedor?

- Una comparativa user-friendly
  - Imaginemos que yo quiero enviar uno de estos artículos desde China hasta España.
  - Si no utilizo contenedores, necesitaré un camión diferente para cada artículo, una grúa diferente, etc.



¿Qué es un contenedor?

- Si utilizo contenedores, me va a dar igual lo que haya dentro del contenedor porque voy a transportarlo todo por igual, gracias a que está estandarizado.







¿Qué es un contenedor?

- A nivel de despliegue:
  - Con contenedores, descargaremos la imagen y la instalaremos en el contenedor con un solo comando.
  - No tendremos problemas de dependencia entre entornos
  - Siempre, independientemente de donde estemos o se utilizarán las mismas versiones
  - Los contenedores están basados en un sistema de capas (donde la capa inferior es el SO Linux)

Ventajas de utilizar contenedores

- En la administración tradicional de servidores de aplicaciones nos encontramos con una serie de tareas que se repiten constantemente en cada máquina: instalación del sistema operativo, actualizaciones y parches necesarios, instalar la última versión de la aplicación y sus dependencias.
  - A medida que aumenta el número de máquinas a gestionar, crece la dificultad de mantenerlas actualizadas con nuevas actualizaciones de seguridad, nuevas versiones de la aplicación, cambios en las dependencias, etcétera.
    - Se trata de proceso bastante propenso a errores.

Ventajas de utilizar contenedores

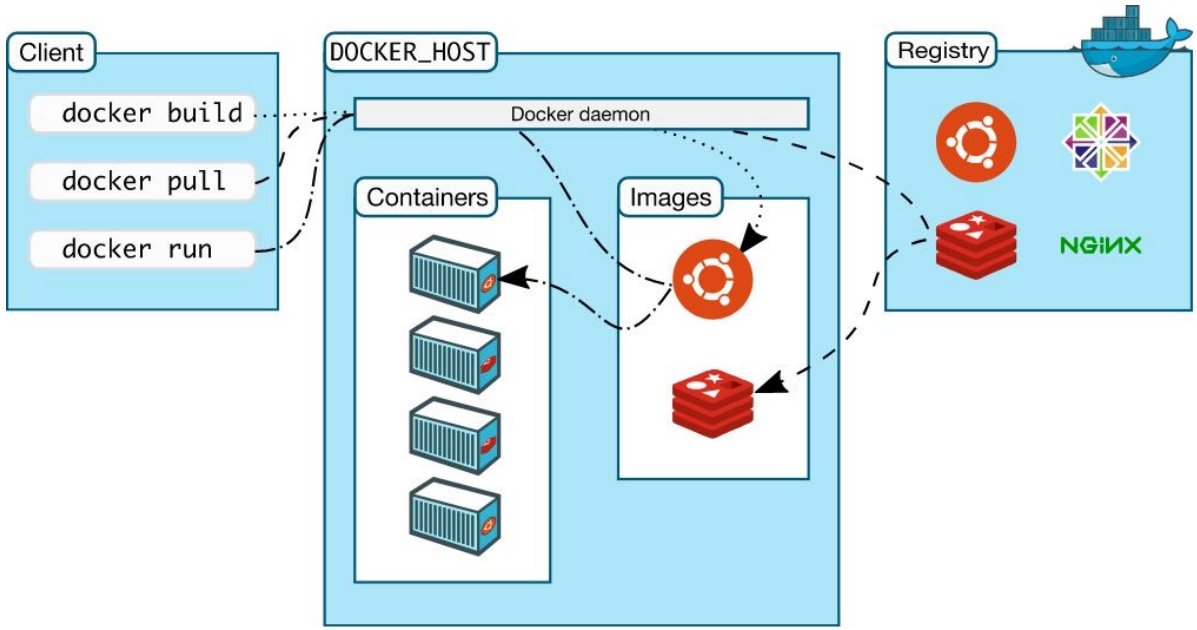
- Con la llegada de Docker tenemos la posibilidad de empaquetar en una imagen todo el código de nuestra aplicación, su sistema operativo, sus dependencias, binarios, ficheros de configuración, y solamente hacer este proceso una vez.

- Si alguno de nuestros contenedores falla y termina, no haría falta sacar a ese servidor del sistema para reparar el problema, sino que simplemente volveremos a desplegar una nueva instancia del contenedor de nuestra aplicación. De este modo, todas las instancias de la aplicación serían copias idénticas que no haría falta configurar individualmente.
  - Esto se puede gestionar automáticamente con orquestadores (U3)

Desventajas

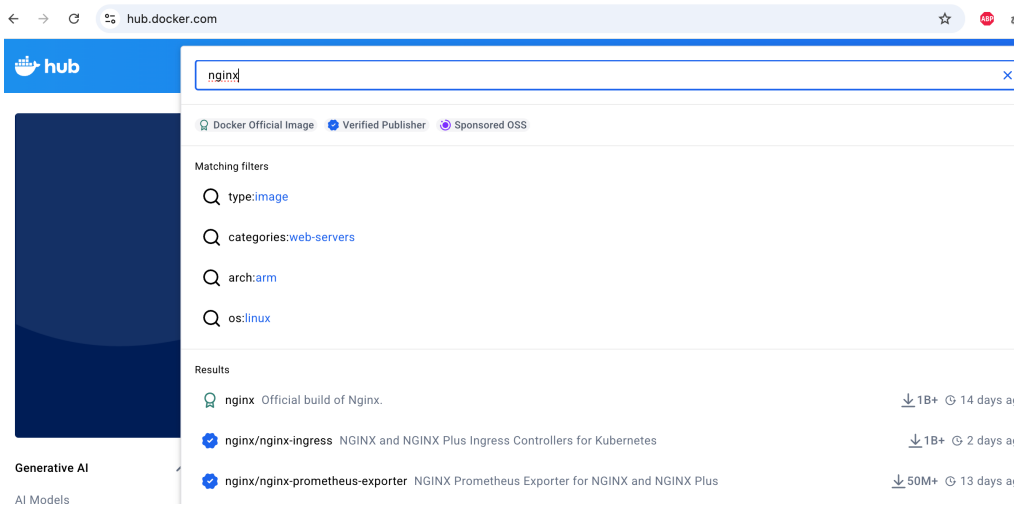
- Órdenes por línea de comandos
  - Existen herramientas gráficas pero aún no tienen una administración avanzada
- No es tan rápido como si fuera nativo.
- La gestión de datos persistentes (volúmenes) y redes puede ser un poco tediosa.

Arquitectura de Docker



¿Dónde se guardan las imágenes?

- Repositorios de imágenes
  - Repositorios privados
  - Repositorios públicos
    - Docker Hub
      - *docker pull*



Instalando Docker

- Se puede instalar directamente la versión Docker Desktop disponible para todas las plataformas.
  - Proporciona una herramienta visual para la gestión de imágenes y contenedores.
  - PROBLEMA-> en un servidor real, esta versión no puede estar instalada ya que no hay interfaz gráfica

- Instalaremos directamente el Docker Engine en nuestro Ubuntu siguiendo el manual:
  - https://docs.docker.com/engine/install/

Instalando Docker

- Una vez instalado, para evitar el uso de sudo en todas las instrucciones relacionadas con Docker, ejecutaremos los pasos indicados en el *Post-Installation Steps.*
  - https://docs.docker.com/engine/install/linux-postinstall/

- Es posible que la instrucción para crear el grupo nos indique que ya existe. Seguiremos igualmente con las siguientes instrucciones, que se basan en añadir el usuario al grupo *Docker.*

Instalando Docker

- En Linux el servicio de Docker se arranca con el inicio del sistema y por tanto las instrucciones están disponibles desde el arranque.

- En Windows y Mac hay que arrancar de forma explícita el servicio ya que realmente, de forma interna, se arranca una virtualización que corre el servicio dentro.
  - Para estos casos, es mejor tener instalado Docker Desktop pero no utilizarlo directamente, solo para que los comandos nos funcionen.

Práctica 1

- Ejecución de un contenedor Ubuntu interactivo (con consola)

  docker run -it ubuntu bash

  cat /etc/os-release

  exit (opcional, se puede seguir jugando)

- Ejecutar *docker ps* y *docker ps -a*
  - ¿Por qué crees que cambia el resultado?

Comandos

- Iremos estudiando los comandos de forma intercalada durante la unidad pero vamos a estudiar algunos de los más importantes que nos serán útiles.

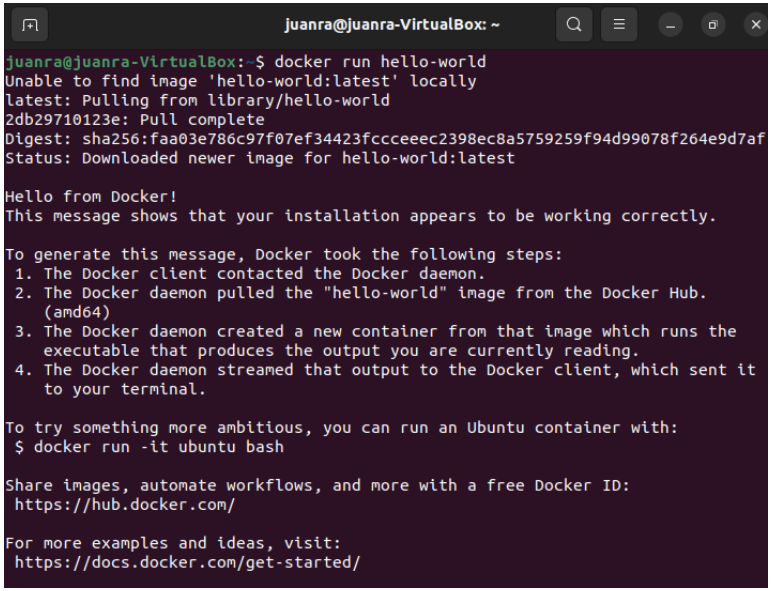
- docker run imageName
  - Busca la imagen en local, si no la encuentra la busca en remoto y la descarga
  - Crea el contenedor
  - Arranca el contenedor

Comandos

- docker run [params] imagen [comandoAlArrancar] [args]

- Entre los parámetros que le podemos pasar, destacamos:
  - -it -> Arrancamos el contenedor en modo interactivo (enlazado al terminal)
  - --name or -n -> le damos un nombre a nuestro contenedor
  - -p -> mapeo de puertos
  - -d -> permite ejecutar el contenedor en segundo plano
  - -e -> permite establecer parámetros de variable
  - --rm -> cuando el contenedor se para, se elimina del sistema

Comandos


juanra@juanra-VirtualBox: ~
juanra@juanra-VirtualBox:~$ docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
2db29710123e: Pull complete
Digest: sha256:faa03e786c97f07ef34423fccceeec2398ec8a5759259f94d99078f264e9d7af
Status: Downloaded newer image for hello-world:latest
Hello from Docker!
This message shows that your installation appears to be working correctly.
To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
(amd64)
3. The Docker daemon created a new container from that image which runs the
executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
to your terminal.
To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash
Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/
For more examples and ideas, visit:
https://docs.docker.com/get-started/

Comandos

- Per a mantener un contenidor corrents i evitar que es tanque (si així ho necessitem i vem que la imatge que gastem fa que es tanque per defecte), cal afegir la següent *coletilla*
  - *tail -f /dev/null*
  - *On tail és un comando a arrancar*

- Per tant se'ns quedaria de la següent forma:
  - docker run -d hello-world tail -f /dev/null
  - Si no posem el -d, ens bloquejarà la terminal i tindrem que obrir altra

Comandos

- docker ps
  - Comprueba los contenedores en ejecución
  - Nos da un nombre para que podamos hacer referencia a él en lugar de utilizar el Container ID (CID)
  - Si añadimos -a al ejecutar el docker ps nos mostrará también los contenedores que han sido detenidos.

```
sergi@ubuntu:~$ docker ps
CONTAINER ID   IMAGE     COMMAND   CREATED         STATUS         PORTS     NAMES
434d318b3771   ubuntu    "bash"    9 seconds ago   Up 7 seconds             stupefied_colden
```

Comandos

- docker start CID/nombre -> arranca el contenedor con ese CID/nombre
- docker stop CID/nombre -> para el contenedor con ese CID/nombre

```
juanra@juanra-VirtualBox:~$ docker ps -a
CONTAINER ID   IMAGE     COMMAND       CREATED          STATUS                          PORTS     NAMES
5ed9b5d3e3f6   ubuntu    "/bin/bash"   41 minutes ago   Exited (129) 37 minutes ago               myUbuntu1
juanra@juanra-VirtualBox:~$ docker start 5ed9b5d3e3f6
5ed9b5d3e3f6
juanra@juanra-VirtualBox:~$ docker stop myUbuntu1
myUbuntu1
juanra@juanra-VirtualBox:~$
```

Comandos

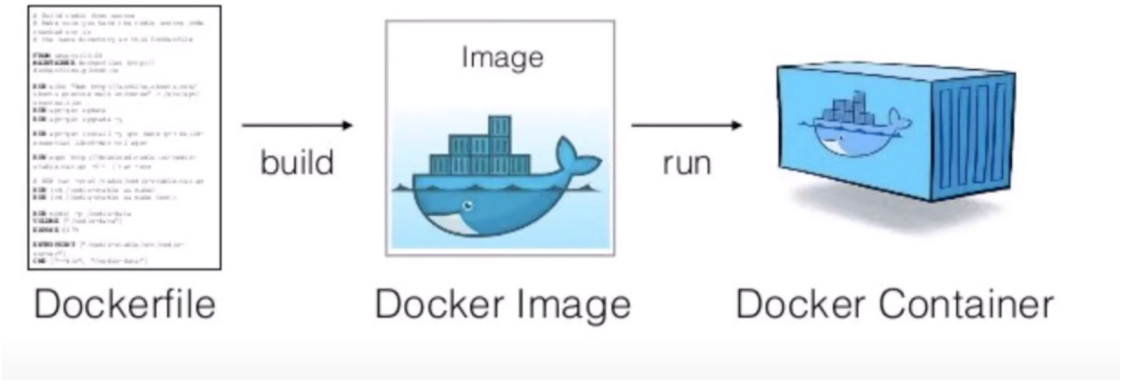
- Otro comando muy utilizado es el *exec* que nos permitirá ejecutar instrucciones en contenedores ya ejecutados. Muy útil para lanzar, por ejemplo, una terminal y poder ejecutar instrucciones sobre ellas:
  - *docker exec [params] CID/nombre instruccion_A_ejecutar*
  - *docker exec -it 12saf1234 /bin/bash*

- En ocasiones /bin/bash no nos conecta con el terminal. Se puede utilizar también *sh.*
  - *docker exec -it 12saf1234 sh*

Creando aplicaciones con docker

- Para poder crear un contenedor de Docker
  - Hemos de crear una imagen de Docker
  - Para ello partiremos de un fichero llamado Dockerfile
    - No puede tener otro nombre
    - Contendrá los servicios y dependencias necesarias para la ejecución de la aplicación
  - Existen otras formas de crear imágenes de Docker que se estudiarán más adelante

Creando aplicaciones con Docker



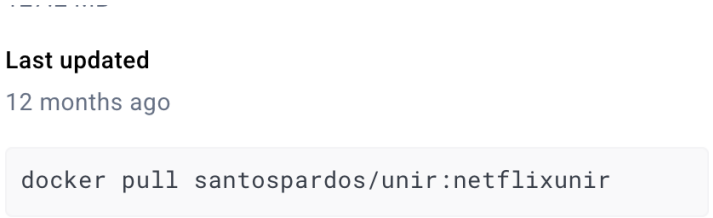
Creando aplicaciones con docker

- Podemos comprobar las imágenes que tenemos descargas en nuestro entorno local ejecutando:
  - docker images
- Para eliminar una imagen descargada:
  - docker rmi imageID

```
juanra@MacBook-Air-de-Juanra ~ % docker images
REPOSITORY          TAG               IMAGE ID       CREATED         SIZE
skills-apache       latest            ce60b43917e9   6 months ago    585MB
<none>              <none>            73e88271b392   6 months ago    518MB
<none>              <none>            208ab3a750a7   6 months ago    515MB
skills-jsonserver   latest            02d74a5f9848   6 months ago    1.13GB
php                 8.3-apache        0c25856f00ab   6 months ago    514MB
<none>              <none>            6374e2c599d2   7 months ago    363MB
mariadb             11.7.2-noble      04a1fed8b56f   7 months ago    363MB
skills-mariadb      latest            849fde4f1fac   7 months ago    363MB
node                22.14.0           60bf98d6baf6   7 months ago    1.12GB
php                 8.3.1             d73fd1a39ba6   20 months ago   530MB
dws/xampplaravel2   latest            16ad019e99ac   21 months ago   1.5GB
dws/xampplaravel    latest            6cbb226b955e   21 months ago   1.5GB
phpstorm_helpers    PS-232.10072.32   4a0a447650b7   23 months ago   4.2MB
<none>              <none>            b2a6493eb1e6   23 months ago   1.25GB
dws/xamppcomposer   latest            dc12898f7d48   23 months ago   1.25GB
dws/xampp           latest            98acc6ac2622   2 years ago     1.24GB
busybox             latest            fc9db2894f4e   2 years ago     4.04MB
juanra@MacBook-Air-de-Juanra ~ %
```

Creando aplicaciones con docker

- Podemos descargar imágenes utilizando el siguiente comando:
  - docker pull nombreImagen
  - docker pull santospardos/unir:netflixunir
    - La imagen la buscamos en docker hub y nos da directamente la instrucción a utilizar para la descarga.




```
juanra@MacBook-Air-de-Juanra ~ % docker pull santospardos/unir:netflixunir
netflixunir: Pulling from santospardos/unir
177e7ef0df69: Pull complete
9bf89f2eda24: Pull complete
350207dcf1b7: Pull complete
a8a33d96b4e7: Pull complete
c0421d5b63d6: Pull complete
f76e300fbe72: Pull complete
af9ff1b9ce5b: Pull complete
d9f072d61771: Pull complete
```

Creando aplicaciones con Docker

- Ejecutamos la aplicación descargada y comprobamos que se esté ejecutando:
  - docker run -d -p 80:80 santospardos/unir:Netflix
  - docker ps

```
juanra@MacBook-Air-de-Juanra ~ % docker run -d -p 80:80 santospardos/unir:netflix
juanra@MacBook-Air-de-Juanra ~ % docker ps
CONTAINER ID   IMAGE                       COMMAND                  CREATED          STATUS          PORTS                NAMES
fe7d4d33ec91   santospardos/unir:netflix   "docker-php-entrypoi…"   45 minutes ago   Up 45 minutes   0.0.0.0:80->80/tcp   nice_noyce
```

Mapeo de puertos

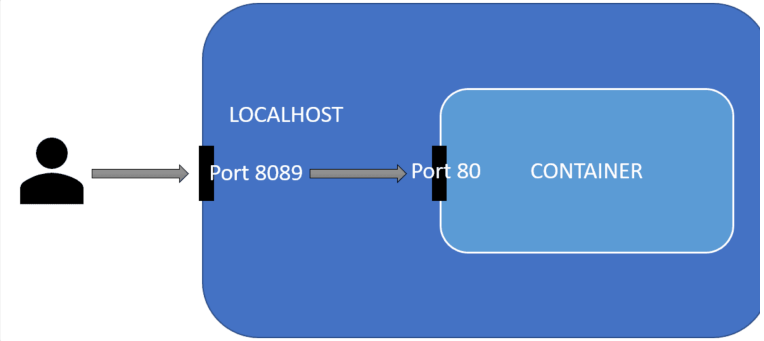
- El contenedor que hemos lanzado contiene un servidor web (con una web).
- Al lanzar el contenedor, hemos mapeado el puerto 80 del contenedor al puerto 80 nuestro
  - Eso significa que si yo accedo al puerto 80 de localhost (o la máquina donde esté el contenedor), éste internamente enviará esa petición al puerto 80.
  - Por tanto, al acceder a http://localhost:80, veré la web

Mapeo de puertos

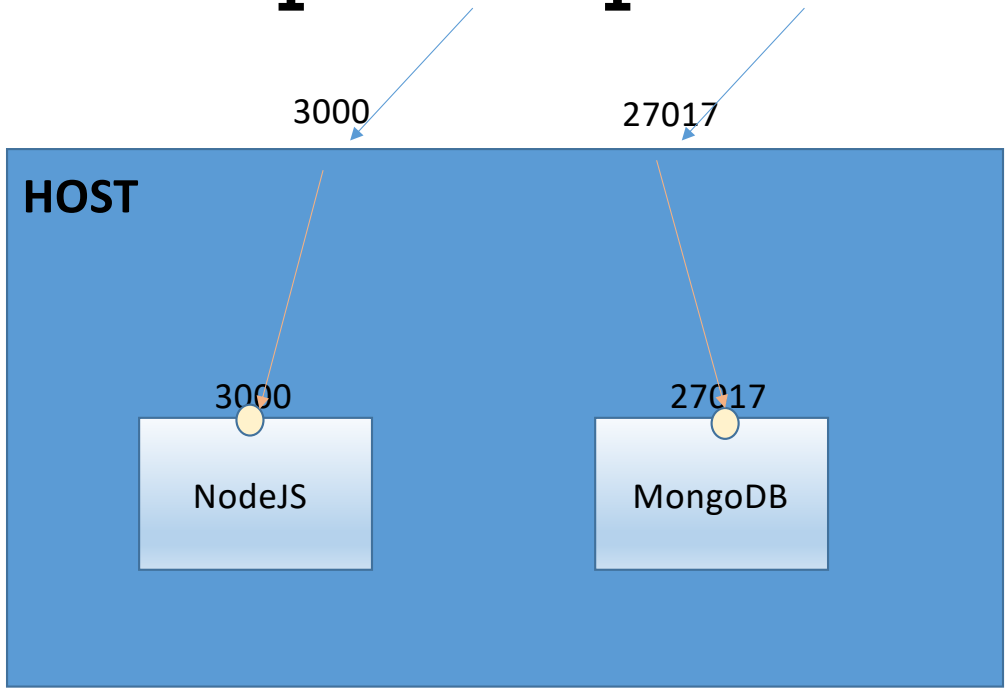
- Si hubiese querido complicarme la vida, podría haber mapeado otro puerto al puerto 80 (recordemos que el 80 es por donde el servidor web sirve el http).
  - docker run -d -p 8089:80 santospardos/unir:Netflix
  - Por tanto, ahora tendría que acceder a localhost:8089
  - Normalmente utilizaremos los mismos puertos de nuestro pc que del contenedor excepto cuando éstos no estén disponibles.

Mapeo de puertos

- El mapeo de puertos solo está disponible para su configuración durante la creación del contenedor y no puede ser cambiado a posterior.



Mapeo de puertos



Comandos

- También podemos crear contenedores sin lanzarlos utilizando *create* en lugar de *run.*

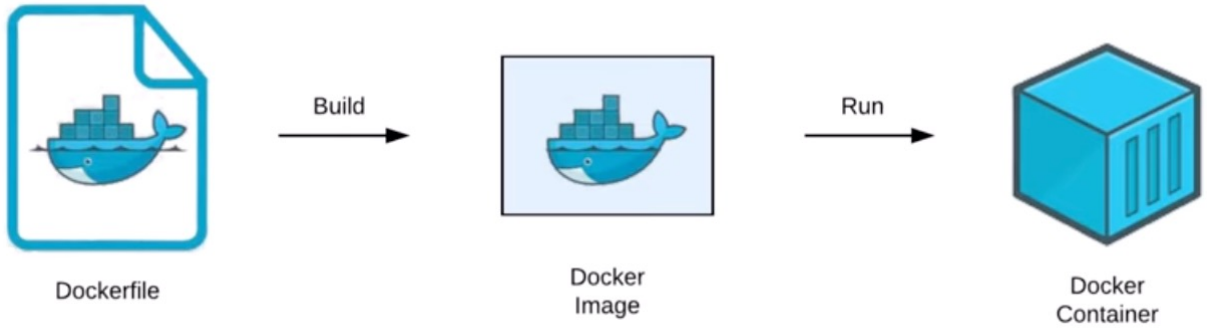
- También podemos eliminar contenedor (importante pararlos previamente):
  - docker stop CID
  - docker rm CID

- https://docs.docker.com/reference/cli/docker/

Dockerfile

- El fichero Dockerfile es el fichero de referencia utilizado para crear una imagen.
  - Es importante que el fichero se llame Dockerfile y que la primera esté en mayúscula.
- Es un fichero clave en la creación del contenedor ya que contiene toda la configuración del contenedor.
- Se recomienda crearlo con Visual Studio Code ya que gracias a sus plugins nos echará una mano con la sintaxis.

Dockerfile



Ejemplo básico

- Para este ejemplo, clonaremos el siguiente repositorio:
  - https://github.com/JuanraCollado/octopus-underwater-app

- Como podéis observar, es una web estática bastante simple, con recursos como CSS, HTML, etc.

- Contiene un fichero Dockerfile, es decir, está preparado para montar un contenedor.

Ejemplo básico

```
juanra@MacBook-Air-de-Juanra ejemplosDocker % git clone https://github.com/JuanraCollado/octopus-underwater-app

Clonando en 'octopus-underwater-app'...
remote: Enumerating objects: 253, done.
remote: Counting objects: 100% (48/48), done.
remote: Compressing objects: 100% (7/7), done.
remote: Total 253 (delta 41), reused 41 (delta 41), pack-reused 205 (from 1)
Recibiendo objetos: 100% (253/253), 259.86 KiB | 663.00 KiB/s, listo.
Resolviendo deltas: 100% (132/132), listo.
juanra@MacBook-Air-de-Juanra ejemplosDocker % cd octopus-underwater-app
juanra@MacBook-Air-de-Juanra octopus-underwater-app % ls -l
total 16
-rw-r--r--   1 juanra  staff   52 23 sep 19:52 Dockerfile
-rw-r--r--   1 juanra  staff  206 23 sep 19:52 README.md
drwxr-xr-x  17 juanra  staff  544 23 sep 19:52 static
juanra@MacBook-Air-de-Juanra octopus-underwater-app % ls static
animation.svg
blogimage-commondeploymentpatternsandhowtousetheminoctopus-2021.png
blogimage-highleveloverviewofoctopusdeploy-2021.png
blogimage-importingadeployabletutorialwithoctopusdeployandazurewebapplications-2021.webp
blogimage-the-ten-pillars-of-pragmatic-deployments-2021.png
boat.svg
clouds.svg
css
index.html
ocean-bed.svg
octopus-logo.svg
profile-andycorrigan.png
profile-matthewcasperson.png
profile-terencewong.png
wave.svg
juanra@MacBook-Air-de-Juanra octopus-underwater-app %
```

Ejemplo básico

- Si abrimos el fichero Dockerfile vemos lo siguiente:



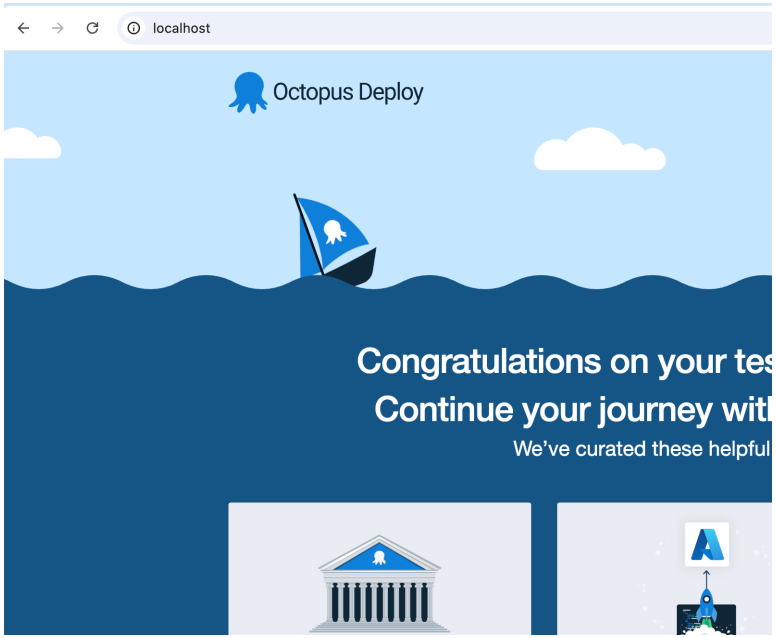
- FROM indica cual será nuestra imagen base
  - En este caso es una imagen de Alpine que contiene ya un servidor nginx instalado
- COPY copia la carpeta que le pasamos como primer argumento en la ruta que le pasamos como segundo argumento
  - En este caso copiaremos la carpeta static (que contiene los ficheros de la web) en la carpeta html de nginx que es la raíz del servidor nginx (como ya sabmos de la unidad anterior).

Ejemplo básico

- Podemos crear la imagen y etiquetarla sin lanzar el contenedor
  - docker build -t nombre_imagen[:nombre_tag] directorioDockerfile
  - docker build -t octopus .
    - Me creará una imagen llamada octopus y le indico que el directorio donde se encuentra el Dockerfile es el mismo en el que me encuentro (.)

```
juanra@MacBook-Air-de-Juanra octopus-underwater-app % docker build -t octopus .
[+] Building 4.3s (8/8) FINISHED                                  docker:desktop-linux
 => [internal] load build definition from Dockerfile                              0.0s
 => => transferring dockerfile: 328B                                              0.0s
 => [internal] load metadata for docker.io/library/nginx:alpine                   1.9s
```

Ejemplo básico



Práctica 2

- Desde línea de comandos, crea una contenedor llamado *practica2* y basado en la imagen de nginx que está basada en alpine pero no lo ejecutes. Además, el puerto 80 de este contenedor debe estar mapeado al puerto 8080 de tu PC. Puedes encontrar información sobre esta imagen en docker-hub.

Práctica 3

- Crea un contenedor donde la base sea un Ubuntu. Conéctate en modo interactivo y navega por él. Intenta crear un documento de texto utilizando nano o gedit.
  - ¿Por qué crees que ninguno está instalado?
  - ¿Cómo lo podemos instalar?

- Tras realizar esta práctica, se recomienda realizar el ejercicio 1 del PEC2.

Dockerfile

- Podemos darle comandos que queremos que ejecute la imagen cuando se está construyendo(antes que el usuario se conecte) mediante la instrucción RUN
  - Sirve para preparar la imagen: crear directorios, instalar paquetes, copiar archivos dentro de la máquina, etc.
    - RUN mkdir -p /home/miCarpeta
      - El -p crea toda la estructura de carpetas completa
    - RUN apt update
  - IMPORTANTE: esto se ejecuta en el contenedor, no en nuestra máquina

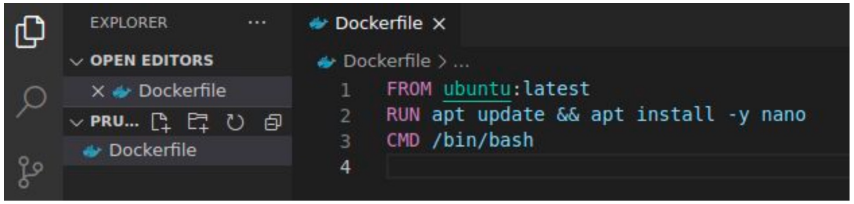
Dockerfile

- Podemos exponer el puerto (o puertos) que utilizará el contenedor
  - EXPOSE 80
  - Sirve como documentación dentro de la imagen para que otros desarrolladores sepan por a qué puerto van a poder conectarse.
  - NO abre ni publica el puerto, seguiremos teniendo que utilizar el -p al crear el contenedor.
  - Piensa en el EXPOSE como un aviso embebido dentro de la propia imagen

Dockerfile

- Por otra parte, tenemos la instrucción CMD que sirve para ejecutar comandos cuando se ejecuta el contenedor
  - La diferencia entre CMD y RUN es que RUN lo hace antes de ejecutar (cuando está preparando la imagen) y CMD cuando ejecutamos el contenedor.
  - CMD bash
  - CMD [“node”,”src/index.js”]
    - El primer parámetro es el programa a ejecutar i los siguientes son los argumentos que le pasamos a este programa.

Dockerfile



- En este caso partimos de la última versión de Ubuntu disponible y ejecutará antes de arrancar el contenedor la instalación del nano (importante el -y) y una vez arranque el contenedor arrancará la terminal
  - Si queremos poder interactuar con el terminal, al arrancar el contenedor será necesario utilizar el parámetro -it.

Dockerfile

- El comando WORKDIR establece el directorio de trabajo por defecto dentro del contenedor para todas las instrucciones que lo siguen en el Dockerfile y para cuando se arranque el contenedor.

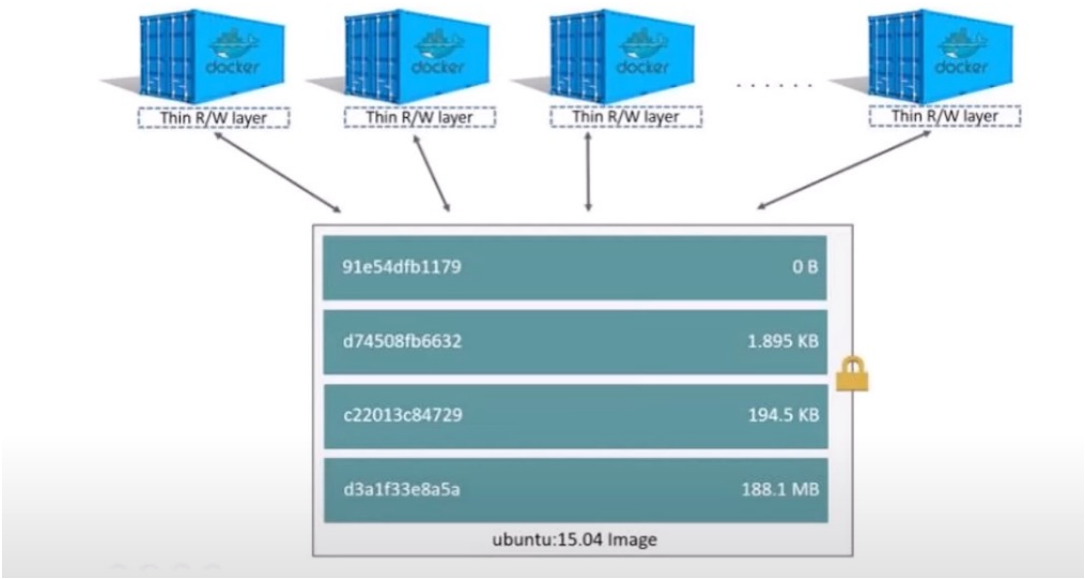
```
FROM ubuntu:20.04
WORKDIR /app
COPY . .        # Se copia el código de la carpeta actual (tu máquina) a /app
RUN make build  # Se ejecuta dentro de /app
CMD ["./myprogram"]  # Al arrancar, se ejecutará desde /app
```

Práctica 4

- Realiza la siguiente práctica guiada del manual oficial de docker:
  - https://docs.docker.com/get-started/workshop/02_our_app/

- La solución a esta práctica está subida en vídeo en Aules. Es importante realizarla correctamente ya que muchos ejemplos posteriores se realizan sobre esta práctica.

Sistema de capas



Sistema de capas

- Las imágenes de Docker están formadas por un conjunto de capas de imagen.
- Cada capa depende de la capa subyacente.
- Cada capa solo incluye los campos respecto a la capa anterior.
- Cuando creamos un fichero Dockerfile, casi cada una de las líneas generará una de las capas.
  - Depende del tipo de instrucción

Sistema de capas

```
FROM ubuntu:20.04         # Crea una capa base
RUN apt-get update         # Nueva capa
RUN apt-get install -y curl  # Nueva capa
COPY . /app                # Nueva capa
CMD ["bash"]               # NO crea capa de FS
```

- Podemos ver el historial de una imagen ejecutando *docker history nombreImagen*

Sistema de capas

- Las capas son inmutables
  - Una vez creadas no se modifican.
  - Si cambiamos algo, se regenera todo

- Cada capa añade tamaño a nuestra imagen.
  - Es buena práctica agrupar las instrucciones que se pueda

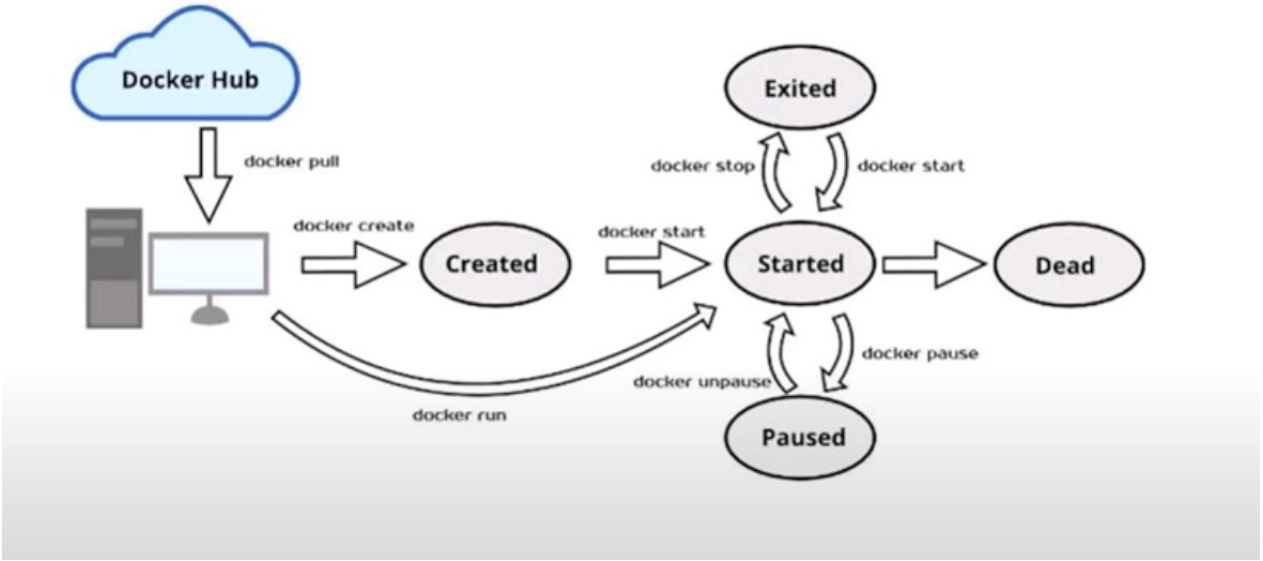
```
RUN apt-get install -y build-essential
RUN apt-get remove -y build-essential
```

```
RUN apt-get update && apt-get install -y build-essential \
    && make app \
    && apt-get remove -y build-essential && apt-get clean \
    && rm -rf /var/lib/apt/lists/*
```

Sistema de capas

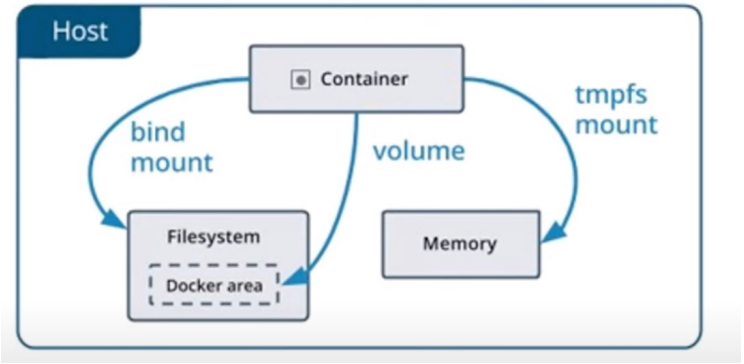
- Si varias imágenes tienen capas comunes, Docker solo las descarga en tu PC una vez.
  - Si tenemos un *FROM node:18* en la definición de 3 Dockerfiles, éste solo lo descargará la primera vez
- Una imagen está compuesta por capas de solo lectura
- Cuando arranca el contenedor Docker añade una nueva capa de lectura/escritura para poder escribir y modificar el contenedor sin modificar la imagen.
  - Cuando paramos/borramos el contenedor, se eliminar esta capa y la imagen sigue intacta.

Ciclo de vida de un contenedor



Almacenamiento en Docker

- Hasta ahora, todos los datos están persistidos dentro del contenedor.
- Es importante poder tener estos datos en la máquina física del contenedor para trabajar de forma cómoda
- Para ello utilizamos lo que llamamos volúmenes



Almacenamiento en Docker

- Hasta ahora, todos los datos están persistidos dentro del contenedor.
- Si no queremos perder estos datos al eliminar el contenedor, es importante persistirlos fuera de éste.
- Para ello utilizamos lo que llamamos volúmenes, que son directorios compartidos entre el host (el pc) y el contenedor.

Almacenamiento en Docker

- Los volúmenes se pueden configuran durante la creación de los contenedores utilizando el parámetro *-v.*
- A este tipo de enlace de los volúmenes con el contenedor se le llama ***Bind Mount*** y es ideal, por ejemplo, para sincronizar código.
  - *docker run -v /home/juanra/myApp:/app ubuntu:latest*
  - La carpeta */home/juanra/myApp* se montaría dentro de la carpeta /app del contenedor de forma que todo lo que hay en la carpeta de mi PC se ve también en el contenedor y viceversa.
  - También se pueden utilizar comandos:
    - docker run -d -p 8080:80 -v $(pwd)/html:/usr/share/nginx/html nginx

Almacenamiento en Docker

- Los volúmenes pueden ser utilizados como en el ejemplo anterior (**Bind mounts**) o se les puede dar un nombre (**Managed Volumes**).

- En el caso de los **Managed Volumes**, no le indicamos en qué carpeta del host se guardan los datos, sino que el propio Docker se encargará de la gestión:
  - Este es el mejor uso para persistir datos (bases de datos, por ejemplo)
  - La carpeta donde guarda los datos suele ser /var/lib/docker/volumes

Almacenamiento en Docker

- Para asignar un nombre a un volumen, primer se ha de crear el volumen y después se utiliza:
  - *docker volumen create miVolumenPropio*

- Y después se utiliza en la creación:
  - *docker run -d -v miVolumenPropio:/var/lib/mysql mysql*
  - En este ejemplo estaríamos persistiendo todos los datos almacenados en la base de datos mysql que contiene el contenedor.

Práctica 5

- Crea y ejecuta un contenedor interactivo con Ubuntu con nombre “sinVolumen”
- Dentro del contenedor, crea una carpeta llamada ”datos” y dentro crea un fichero llamado “mensaje.txt” con el mensaje “hola desde el contenedor”
  - Para ello no es necesario instalar editor de texto, podemos utilizar la redirección de salida estándar >
- Sal del contenedor
- Elimina el contenedor
- Continua el siguiente diapositiva

Práctica 5

- Vuelve a lanzar un nuevo contenedor con Ubuntu.
  - ¿Qué ha pasado con los datos que tenías?
  - Efectivamente, los datos del contenedor, no persisten una vez eliminamos el contenedor.
- Elimina el contenedor nuevo.
- Crea un nuevo volumen llamado mi_volumen
  - *docker volume create mi_volumen*
- Ejecuta un contenedor Ubuntu con el volumen montado en /datos con nombre “con_volumen”
  - docker run -it --name con_volumen -v mi_volumen:/datos ubuntu bash
- Continua…

Práctica 5

- Crea el fichero igual que hemos hecho antes
  - echo "Persistencia con volúmenes" > /datos/nota.txt

- Sal y elimina el contenedor
- Lanza otro contenedor con **el mismo volumen** y verifica si el fichero existe.
  - docker run -it -v mi_volumen:/datos ubuntu bash
  - cat /datos/nota.txt

Práctica 5.1

- Haz lo mismo pero en lugar de utilizando Bind Mounts:
  - Previamente habría que crear en nuestro pc un directorio para poder enlazarlo (en mi caso se llama mi_app_datos)
  - docker run -it --name con_bind -v $(pwd)/mi_app_datos:/datos ubuntu bash

Práctica 6

- Realiza la siguiente práctica guiada del manual oficial de docker:
  - https://docs.docker.com/get-started/workshop/05_persisting_data/
  - Puedes ir directamente a la sección *Create a volumen and start the container*

- La solución a esta práctica está subida en vídeo en Aules.

Práctica 6

- Crea y ejecuta un contenedor interactivo con Ubuntu con nombre “sinVolumen”
- Dentro del contenedor, crea una carpeta llamada ”datos” y dentro crea un fichero llamado “mensaje.txt” con el mensaje “hola desde el contenedor”
  - Para ello no es necesario instalar editor de texto, podemos utilizar la redirección de salida estándar >
- Sal del contenedor
- Elimina el contenedor
- Continua el siguiente diapositiva

Almacenamiento en Docker

- Para listas los volúmenes existentes
  - docker volume ls

- Para comprobar donde se guarda el volumen
  - docker inspect volumeName
  - Nos devuelve un json donde *mountpoint* contiene la ruta del volumen.

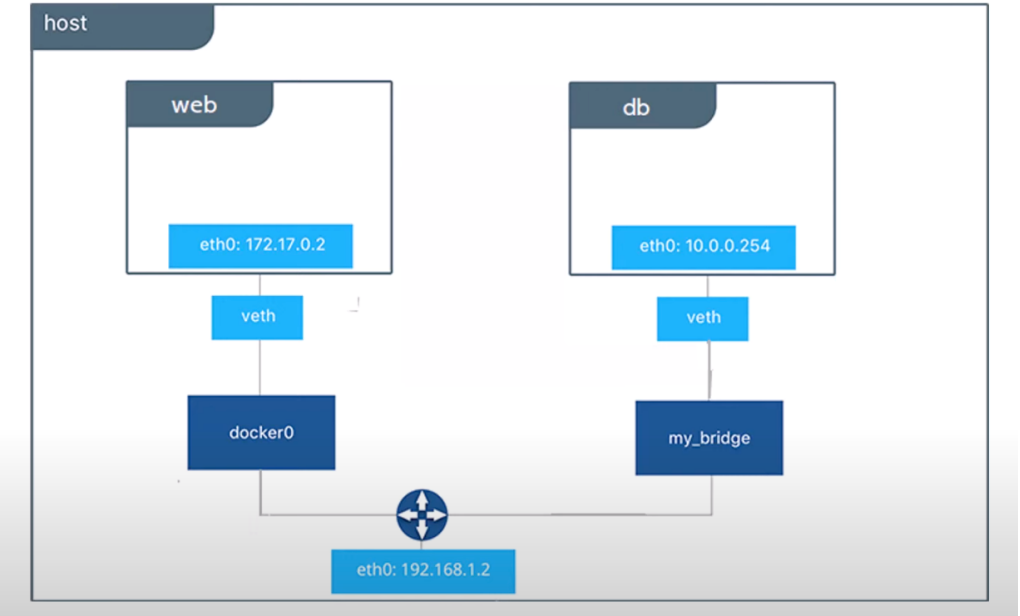
- Para eliminar volúmenes existentes
  - docker volume rm volumeName

Redes en Docker

- Los contenedores pueden necesitar conectarse entre sí.
  - Por ejemplo, puede tener en un contenedor la aplicación web y en otro contenedor la base de datos.

- Docker provee un sistema un sistema de red aislado que evita conflictos con la red del pc y el resto de redes.

Redes en Docker



Juanra Collado

IES MARIA ENRÍQUEZ

Redes en Docker

- Para ver las redes que tenemos
  - *docker network ls*

```
juanra@MacBook-Air-de-Juanra ~ % docker network ls
NETWORK ID     NAME              DRIVER    SCOPE
e68807523441   bridge            bridge    local
473295d8de79   host              host      local
fcd5d787f870   none              null      local
```

- Tenemos tres tipos de redes por defecto:
  - Bridge: Controlador por defecto. Permite comunicación entre dos contenedores independientes. (Modo por defecto)
  - Host: Permite eliminar el aislamiento de red entre contenedor y anfitrión.
  - None: Deshabilita las redes del contenedor

Redes en Docker

- Por defecto, los contenedores se crean en modo bridge.

- La única diferencia entre crear una red a mano y dejarla por defecto es que permite la resolución de nombres.
  - Por defecto, solo podemos hacer referencia a otras máquinas por su IP.

- Para crear una red nueva:
  - *docker network create -d tipoRed nombreRed*
  - *docker network create -d bridge miRed*

Redes en Docker

- Para crear el contenedor con la red que hemos creado:
  - docker run -d --network miRed nginx

- Las redes ofrecen muchas más opciones (filtrado de MAC's, servicios DNS, etc) pero están fuera del alcance de este curso.

- Para más información sobre las redes en Docker:
  - https://docs.docker.com/engine/network/

Práctica 7

- En esta práctica vamos a crear un servidor Node.js y una base de datos MongoDB en contenedores separados y conectarlos a través de una red propia a través de sus nombres (no por IP).

- Creamos la red:
  - *docker network create app-net*

- Levantamos el contenedor de MongoDB:
  - *docker run -d --name mongo --network app-net -e MONGO_INITDB_ROOT_USERNAME=root -e MONGO_INITDB_ROOT_PASSWORD=example mongo*
  - Recuerda que -e era para establecer variables de entorno. En este caso creamos el usuario root con usuario password de la base de datos.

Práctica 7

- En esta práctica vamos a crear un servidor Node.js y una base de datos MongoDB en contenedores separados y conectarlos a través de una red propia a través de sus nombres (no por IP).

- Creamos la red:
  - *docker network create app-net*

- Levantamos el contenedor de MongoDB:
  - *docker run -d --name mongo --network app-net -e MONGO_INITDB_ROOT_USERNAME=root -e MONGO_INITDB_ROOT_PASSWORD=example mongo*
  - Recuerda que -e era para establecer variables de entorno. En este caso creamos el usuario root con usuario password de la base de datos.

Práctica 7

- Para acceder por red a este contenedor, como utiliazamos una red propia, podremos utilizar su nombre de contenedor, en este caso *mongo.*

- Vamos a preparar la aplicación de Node.js. Para ello crearemos una carpeta *node-app* con dos archivos:
  - *package.json*
  - *app.js*

- Podéis descargarlos también del siguiente repositorio:
  - https://github.com/JuanraCollado/node-app

Práctica 7

package.json

```json
{
  "name": "docker-mongo-test",
  "version": "1.0.0",
  "main": "app.js",
  "dependencies": {
    "express": "^4.18.2",
    "mongodb": "^6.5.0"
  }
}
```

app.js

```js
const express = require("express");
const { MongoClient } = require("mongodb");

const app = express();
const port = 3000;

// URL de conexión → usamos el nombre del contenedor "mongo"
const url = "mongodb://root:example@mongo:27017";
const client = new MongoClient(url);

app.get("/", async (req, res) => {
  try {
    await client.connect();
    const db = client.db("testdb");
    const collection = db.collection("messages");

    // Insertar un documento de prueba
    await collection.insertOne({ text: "Hola desde Node + Mongo en Docker" });

    // Leer documentos
    const docs = await collection.find().toArray();

    res.json(docs);
  } catch (err) {
    res.status(500).send(err.message);
  }
});

app.listen(port, () => {
  console.log(`Servidor Node escuchando en http://localhost:${port}`);
});
```

Juanra Collado

IES MARIA ENRÍQUEZ

Práctica 7

- Nos centraremos en el código js donde conecta con la base de datos:
  - Linea 8:
    - *const url = "mongodb://root:example@mongo:27017";*
    - Se utiliza el nombre:password seteado al crear el contenedor y en lugar de poner la dirección IP tras la @, se pone el nombre del contenedor (utilizamos DNS).
    - IMPORTANTE: ¿Por qué no hemos mapeado el puerto 27017?
      - El puerto 27017 es el puerto por defecto de MongoDB y éste es utilizado por entre NodeJS y MongoDB **internamente** y por tanto, no necesita ser expuesto al usuario.
      - También se podría haber omitido y funcionaría igual ya que el 27017 es el puerto por defecto de Mongo.
        - *const url = "mongodb://root:example@mongo";*

Práctica 7

- Vamos a crear un fichero Dockerfile que parta de una imagen que ya tiene el servidor Node y que copie los ficheros en la carpeta del servidor.

- Tras esto, expone el puerto 3000 (por el que nos conectaremos a través de la web)

- Arrancamos la aplicación

Práctica 7

Dockerfile 1, U

Dockerfile > ...

```
1   FROM node:18 (last pushed 4 months ago)
2
3   # Crear directorio de trabajo
4   WORKDIR /usr/src/app
5
6   # Copiar package.json e instalar dependencias
7   COPY package.json ./
8   RUN npm install
9
10  # Copiar el resto de archivos
11  COPY . .
12
13  # Exponer puerto
14  EXPOSE 3000
15
16  # Comando de arranque
17  CMD ["node", "app.js"]
```

Juanra Collado

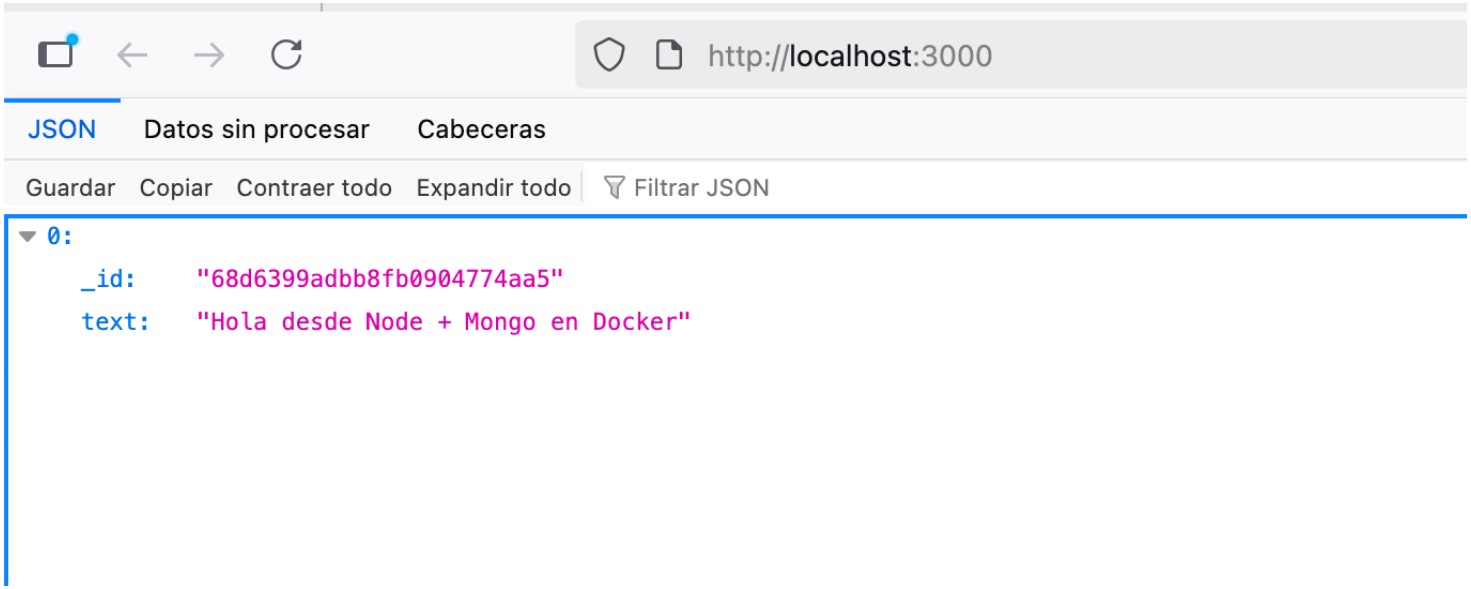
IES MARIA ENRÍQUEZ

Práctica 7

- Construimos la imagen:
  - *docker build -t node-mongo-app .*

- Ejecutamos el contenedor:
  - *docker run -d --network app-net -p 3000:3000 node-mongo-app*
    - --network nos asegura que está en la misma red
    - -p expone el puerto del servidor Node para poder acceder a la web

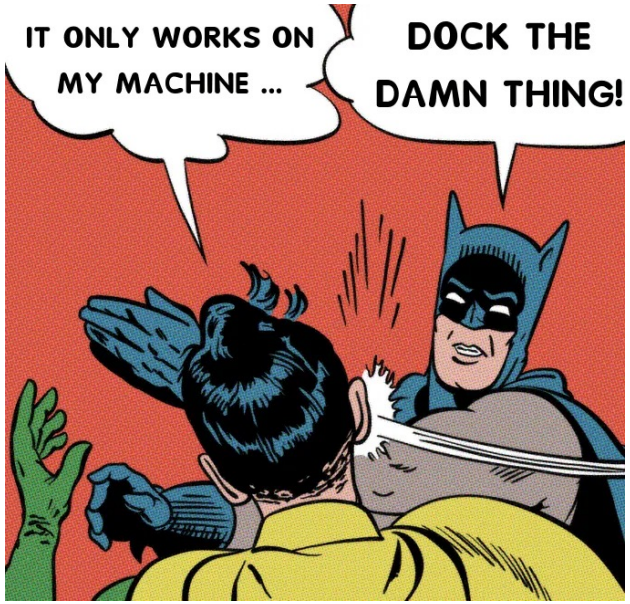
Práctica 7

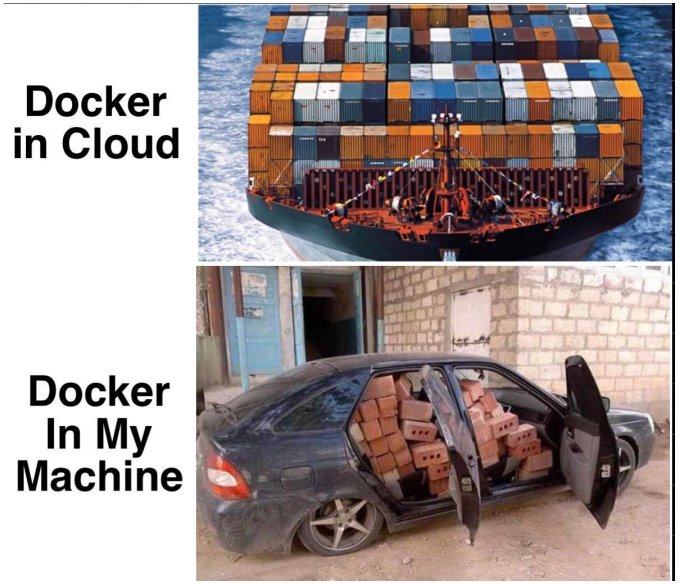


Juanra Collado

IES MARIA ENRÍQUEZ

Algunos memes de Docker





Algunos memes de Docker







¿Dudas?

